

MODULE 6 · SCHEDULING, RESOURCES & SCALING

ResourceQuota & LimitRange

Namespace-wide caps, and the per-container defaults that make them work

Per-container limits don't stop a namespace being emptied

Lab 6.2 gave every container its own requests/limits ceiling. But nothing stops fifty Pods sharing one namespace from together claiming every CPU core and byte of memory the cluster has — per-container caps say nothing about the **total**.

And nothing forces anyone to set requests/limits at all. A careless resourceless Pod slips past a per-container rule with nothing to enforce.

BY THE END YOU CAN

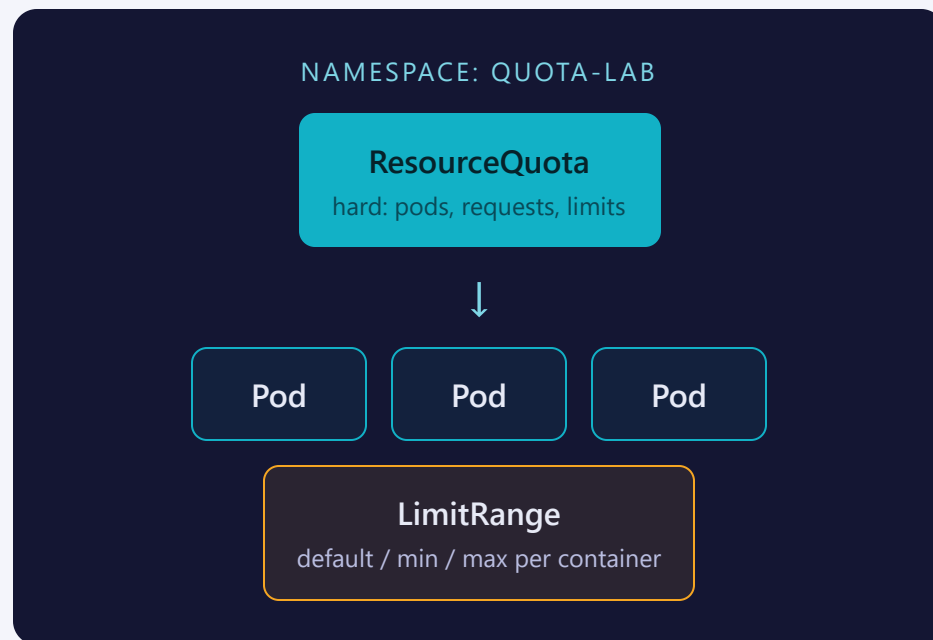
- Cap a namespace's total CPU/memory/object counts
- Set per-container default, min, and max
- Explain why a resourceless Pod gets rejected
- Trace the admission order that can save it

CONCEPT

Two guardrails, two different jobs

- **ResourceQuota** — a ceiling on **totals** in one namespace: the sum of every container's requests, the sum of every limit, and object counts (Pods, Services, PVCs...)
- **LimitRange** — a rule **per container**: fills in a default when nothing is set, and enforces a min/max every container must fit inside

Neither replaces the other. A quota with no LimitRange just blocks resourceless Pods outright; a LimitRange with no quota bounds containers but never caps the namespace total.



Same guardrail idea, different scope

RESOURCEQUOTA	LIMITRANGE
Scope: the whole namespace	Scope: each container
Constrains sum of requests/limits + object counts	Constrains default, min, and max per container
Resourceless Pod → nothing to sum → rejected	Resourceless Pod → <code>defaultRequest</code> / <code>default</code> fill it in
Checked against the namespace's running total	Checked against a single container's own spec

MENTAL MODEL

ResourceQuota polices the sum. LimitRange polices — and pre-fills — every individual container.

The ResourceQuota: hard caps on the whole namespace

```
# resourcequota.yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: namespace-quota
  namespace: quota-lab
spec:
  hard:
    pods: "10"           # object count
    requests.cpu: "2"     # sum of ALL requests
    requests.memory: 4Gi
    limits.cpu: "4"
    limits.memory: 8Gi    # sum of ALL limits
```

Applied to namespace quota-lab from labs/6.3/; pairs with the LimitRange on the next slide.

The LimitRange: default, min, and max per container

```
# limitrange.yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: quota-lab
spec:
  limits:
  - type: Container
    default:
      cpu: 200m      # if unset
      memory: 256Mi
    defaultRequest:
      cpu: 100m
      memory: 128Mi
    max:
      cpu: 500m      # ceiling
      memory: 512Mi
    min:
      cpu: 50m        # floor
      memory: 64Mi
```

Both objects target the same namespace, quota-lab — apply them together from labs/6.3/.

ResourceQuota has a shortcut; LimitRange doesn't

IMPERATIVE — FAST, ONE-OFF

```
kubectl create quota namespace-quota --hard=pods=10,requests.cpu=2
```

No equivalent — there's no `kubectl create limitrange`

DECLARATIVE — VERSIONABLE, REPEATABLE

```
kubectl apply -f resourcequota.yaml
```

`kubectl apply -f limitrange.yaml` — the only way to create one

THE BRIDGE TRICK

Generate the ResourceQuota YAML from the imperative command, then hand-write the LimitRange next to it:

```
$ kubectl create quota namespace-quota -n quota-lab \  
  --hard=pods=10,requests.cpu=2 --dry-run=client -o yaml > resourcequota.yaml
```

SEE IT

Defaults get filled in before the quota adds them up



`kubectl describe resourcequota` shows the **Used** column climb toward **Hard** as each admitted Pod's — possibly defaulted — requests are added; `kubectl get pod ... -o jsonpath='{.spec.containers[0].resources}'` shows a Pod that now carries values it never declared.

Why a resourceless Pod is rejected — and what saves it

Once a ResourceQuota puts a hard cap on `requests.cpu/requests.memory`, Kubernetes must have something to add for every Pod. A Pod with no resources block has nothing to sum — with no LimitRange in front, the API server **rejects it outright**.

A LimitRange runs earlier in admission. It mutates the Pod first, injecting `default/defaultRequest` into any container missing them — **before** the ResourceQuota controller ever inspects it.

IN PRACTICE

A Pod requesting `limits.cpu: 5` violates **both** the LimitRange max (500m) and the ResourceQuota total (2 CPU).

LimitRange runs first, so the error you see is its 500m violation — the ResourceQuota check never runs.

Where people trip

- **Quota without LimitRange.** If the quota caps `requests.cpu/memory`, every Pod must declare them explicitly — no `LimitRange` means resourceless Pods are simply rejected.
- **No imperative LimitRange.** There's no `kubectl create limitrange` shortcut — it's YAML or nothing.
- **Quota counts are eventually consistent.** A burst of creates can momentarily let one extra Pod through before rejection kicks in — the final count still lands on the hard cap.
- **Blaming the wrong layer.** A rejection may show the `LimitRange`'s min/max error even when you were really testing the quota — `LimitRange` gets first look.

Commands to keep at hand

```
$ kubectl create quota NAME --hard=pods=10,requests.cpu=2 # imperative, quota only
$ kubectl describe resourcequota NAME -n NS             # Used vs Hard
$ kubectl describe limitrange NAME -n NS                # default / min / max in effect
$ kubectl get resourcequota,limitrange -n NS            # both objects at a glance
$ kubectl get pod NAME -o jsonpath='{.spec.containers[0].resources}' # injected defaults
```

Tip: describe resourcequota before creating more Pods — it tells you exactly how much headroom is left.

RECAP

ResourceQuota & LimitRange in one breath

- ResourceQuota caps namespace totals — requests, limits, object counts
- LimitRange sets per-container default, min, and max
- A resourceless Pod is rejected under a cpu/memory quota — unless a LimitRange fills it in first
- Admission order: LimitRange defaults, **then** the ResourceQuota check

HANDS-ON

Now run Lab 6.3
[labs/6.3/](#)

Next: [6.4 — Horizontal Pod Autoscaler](#)